Cambridge Default Journal (), , 1–14

Instituto Tecnológico de Apizaco

ARTICLE

Implementación de un Perceptrón Interactivo para la Simulación de Operadores Lógicos

Diego Iván González Galán, Valeria Zoe Torres Castro, y Uriel Ramírez Hernández

Ver Proyecto en Overleaf | Ver Repositorio GitHub

Introducción

Las redes neuronales artificiales constituyen una de las bases fundamentales del aprendizaje automático, ya que permiten desarrollar modelos capaces de identificar patrones a partir de datos de entrada. Uno de los modelos más sencillos para comprender el funcionamiento de una neurona artificial es el perceptrón, un algoritmo de aprendizaje supervisado que utiliza pesos asociados a sus entradas para generar una salida determinada.

El presente trabajo aborda el desarrollo de un programa en Python capaz de entrenar un perceptrón para resolver diferentes operadores lógicos (AND, OR, NAND y NOR), utilizando sus respectivas tablas de verdad como datos de entrenamiento. Para cada operador se emplean diferentes tasas de aprendizaje (0.01, 0.25, 0.50 y 0.75) y se registra el comportamiento del error durante las épocas de entrenamiento mediante tablas acumuladas y gráficas de convergencia.

1. Problema

Uno de los problemas fundamentales al estudiar redes neuronales artificiales consiste en comprender cómo una neurona modifica sus pesos durante el proceso de aprendizaje y cómo estos cambios influyen en la capacidad del modelo para producir las salidas esperadas.

Las operaciones lógicas representan un escenario sencillo para estudiar este comportamiento, debido a que poseen entradas y salidas perfectamente definidas. Por ejemplo, para la operación AND, solamente se obtiene una salida de 1 cuando ambas entradas son iguales a 1. De manera similar, OR, NAND y NOR cuentan con patrones específicos que pueden utilizarse como datos de entrenamiento.

El problema planteado consiste en desarrollar un programa que permita entrenar un perceptrón utilizando las tablas de verdad de diferentes operadores lógicos, modificando sus pesos mediante una tasa de aprendizaje y determinando si el modelo logra converger dentro de un número determinado de épocas.

Para realizar este proceso, el programa utiliza cuatro tasas de aprendizaje: 0.01, 0.25, 0.50 y 0.75. De esta manera, es posible observar cómo diferentes valores de la tasa de aprendizaje pueden modificar el proceso de entrenamiento.

Otro aspecto importante del problema es la necesidad de visualizar el comportamiento del error. Por ello, el programa registra el error total obtenido durante cada época y posteriormente genera una gráfica de error contra épocas, permitiendo analizar visualmente la convergencia del entrenamiento.

2. Metodología

La metodología utilizada para desarrollar el proyecto se dividió en diferentes etapas: definición del problema, preparación de los datos, implementación del modelo, entrenamiento, registro de resultados y visualización.

2.1 Definición de los datos de entrada

En primer lugar, se establecieron las posibles combinaciones de dos entradas binarias. Para incorporar el sesgo de la neurona, cada combinación incluye una primera entrada cuyo valor es 1 ($x_0 = 1$). El programa utiliza las siguientes combinaciones:

- $[1, 0, 0]$
- $[1, 0, 1]$
- $[1, 1, 0]$
- $[1, 1, 1]$

Estas entradas se almacenan en una matriz mediante NumPy. Posteriormente, se definieron las salidas esperadas correspondientes a cada operador lógico:

- $AND \rightarrow [0, 0, 0, 1]$
- $OR \rightarrow [0, 1, 1, 1]$
- $NAND \rightarrow [1, 1, 1, 0]$
- $NOR \rightarrow [1, 0, 0, 0]$

2.2 Inicialización de los pesos

Para comenzar cada entrenamiento se generan tres pesos iniciales correspondientes al sesgo y a las dos entradas del modelo (w_0, w_1, w_2). Estos valores se generan de manera aleatoria mediante NumPy, utilizando valores comprendidos entre 0 y 0.99. También se utiliza una semilla aleatoria asociada al número de corrida para garantizar reproducibilidad.

2.3 Proceso de entrenamiento

El entrenamiento se realiza mediante un ciclo de épocas. En cada época se recorren las cuatro combinaciones de entrada y se calcula el valor neto mediante el producto punto entre las entradas y los pesos.

Posteriormente, se aplica una función de activación binaria. Si el valor neto es mayor o igual a cero, la salida producida es 1; en caso contrario, la salida es 0. El error se obtiene comparando la salida deseada con la salida generada:

$$Error = SalidaDeseada - SalidaPredicha \tag{1}$$

Cuando existe un error diferente de cero, los pesos se actualizan utilizando la tasa de aprendizaje (η):

$$w_i = w_i + \eta \cdot Error \cdot x_i \tag{2}$$

El error absoluto se acumula para obtener el error total de cada época.

2.4 Evaluación de la convergencia

Después de procesar las entradas de una época, el error total se almacena en un historial. Si el error total llega a cero, el programa considera que el modelo ha convergido y finaliza el entrenamiento antes de alcanzar necesariamente el número máximo de épocas establecido.

2.5 Registro y visualización

Los resultados de cada corrida se almacenan utilizando un DataFrame de Pandas para mostrar una tabla acumulada de resultados. Además, se genera una gráfica donde el eje horizontal representa las épocas y el eje vertical representa el error total, aplicando `make_interp_spline` de SciPy para suavizar las curvas cuando existen suficientes datos.

2.6 Interacción con el usuario

Se implementó un menú interactivo en consola que permite al usuario seleccionar el operador (AND, OR, NAND, NOR) e indicar el límite máximo de épocas, incorporando validaciones de entrada.

3. Instrumentación

Para el desarrollo del proyecto se utilizaron las siguientes herramientas:

- **Python:** Lenguaje principal utilizado con las bibliotecas NumPy (matrices), Pandas (tablas), Matplotlib (gráficas) y SciPy (`make_interp_spline`).
- **Visual Studio Code:** Entorno de desarrollo para escribir, probar y depurar el código.
- **GitHub:** Plataforma de control de versiones y almacenamiento del código fuente.
- **LaTeX:** Herramienta de tipografía utilizada para elaborar la documentación técnica con formato académico.

4. Resultados y Análisis

El programa permite realizar hasta cuatro corridas por cada operador lógico utilizando las tasas de aprendizaje: 0.01, 0.25, 0.50 y 0.75.

4.1 Operador AND

4.1.1 Corrida 1 (η = 0.01)

TABLA DE RESULTADOS ACUMULADA - OPERADOR: AND

Corrida	Tasa (η)	Pesos Inic [w0, w1, w2]	Pesos Fin [w0, w1, w2]	Épocas Límite	Épocas Usadas	Estado
1	0.01	[0.2917, 0.5253, 0.1896]	[-0.2383, 0.2253, 0.0596]	50	31	Convergió

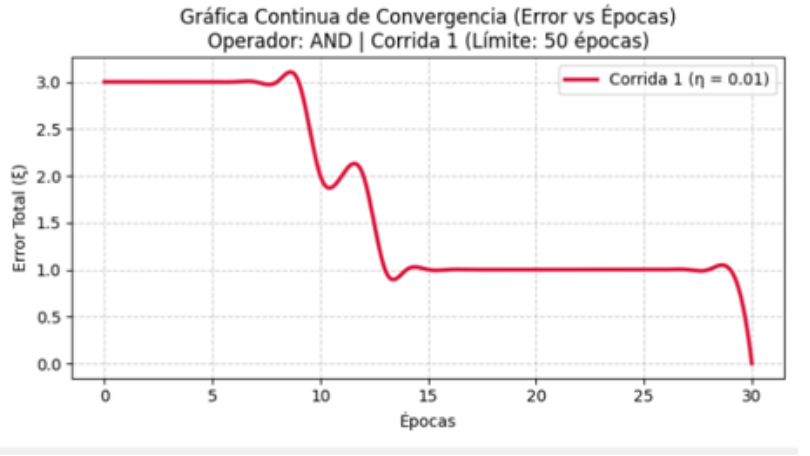
- **Pesos Iniciales:** $w_0 = 0.2917, w_1 = 0.5253, w_2 = 0.1896$
- **Pesos Finales:** $w_0 = -0.2383, w_1 = 0.2253, w_2 = 0.0596$
- **Épocas:** Usó 31 de un límite de 50. Estado: Convergió.

4.1.2 Corrida 2 (η = 0.25)

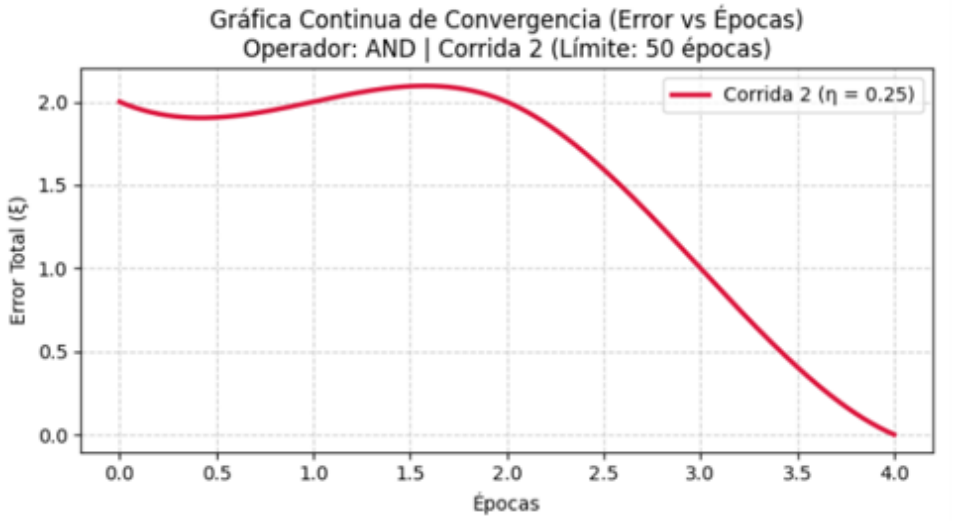
TABLA DE RESULTADOS ACUMULADA - OPERADOR: AND

Corrida	Tasa (η)	Pesos Inic [w0, w1, w2]	Pesos Fin [w0, w1, w2]	Épocas Límite	Épocas Usadas	Estado
1	0.01	[0.2917, 0.5253, 0.1896]	[-0.2383, 0.2253, 0.0596]	50	31	Convergió
2	0.25	[0.0382, 0.7723, 0.0918]	[-0.7118, 0.5223, 0.3418]	50	5	Convergió

- **Pesos Iniciales:** $w_0 = 0.0382, w_1 = 0.7723, w_2 = 0.0918$
- **Pesos Finales:** $w_0 = -0.7118, w_1 = 0.5223, w_2 = 0.3418$
- **Épocas:** Usó 5 de un límite de 50. Estado: Convergió.



Al incrementar η a 0.25, los ajustes por cada error son de mayor magnitud. Encuentra el hiperplano que separa las clases en solo 5 épocas.



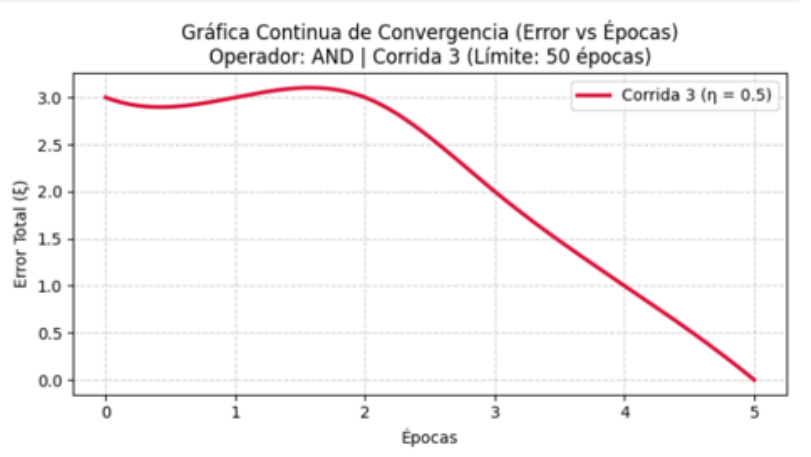
4.1.3 Corrida 3 (η = 0.50)

- **Pesos Iniciales:** $w_0 = 0.6690, w_1 = 0.0443, w_2 = 0.3399$
- **Pesos Finales:** $w_0 = -1.3310, w_1 = 1.0443, w_2 = 0.3399$
- **Épocas:** Usó 6 de un límite de 50. Estado: Convergió.

Inicia en la época 0 con un error total de 3.0. A partir de la época 2, la gráfica muestra un descenso constante hasta alcanzar 0.0 en la época 6.

TABLA DE RESULTADOS ACUMULADA - OPERADOR: AND

Corrida	Tasa (η)	Pesos Inic [w0, w1, w2]	Pesos Fin [w0, w1, w2]	Épocas Límite	Épocas Usadas	Estado
1	0.01	[0.2917, 0.5253, 0.1896]	[-0.2383, 0.2253, 0.0596]	50	31	Convergió
2	0.25	[0.0382, 0.7723, 0.0918]	[-0.7118, 0.5223, 0.3418]	50	5	Convergió
3	0.50	[0.669, 0.0443, 0.3399]	[-1.331, 1.0443, 0.3399]	50	6	Convergió



4.1.4 *Corrida 4 (η = 0.75)*

TABLA DE RESULTADOS ACUMULADA - OPERADOR: AND

Corrida	Tasa (η)	Pesos Inic [w0, w1, w2]	Pesos Fin [w0, w1, w2]	Épocas Límite	Épocas Usadas	Estado
1	0.01	[0.2917, 0.5253, 0.1896]	[-0.2383, 0.2253, 0.0596]	50	31	Convergió
2	0.25	[0.0382, 0.7723, 0.0918]	[-0.7118, 0.5223, 0.3418]	50	5	Convergió
3	0.50	[0.669, 0.0443, 0.3399]	[-1.331, 1.0443, 0.3399]	50	6	Convergió
4	0.75	[0.2568, 0.0429, 0.5762]	[-1.9932, 1.5429, 0.5762]	50	6	Convergió

- **Pesos Iniciales:** $w_0 = 0.2568, w_1 = 0.0429, w_2 = 0.5762$
- **Pesos Finales:** $w_0 = -1.9932, w_1 = 1.5429, w_2 = 0.5762$
- **Épocas:** Usó 6 de un límite de 50. Estado: Convergió.

Debido a la alta tasa ($\eta = 0.75$), en el primer paso presenta un ligero sobretiro (overshoot), subiendo a 3.0 en la época 2 antes de descender linealmente hasta 0 en la época 6.

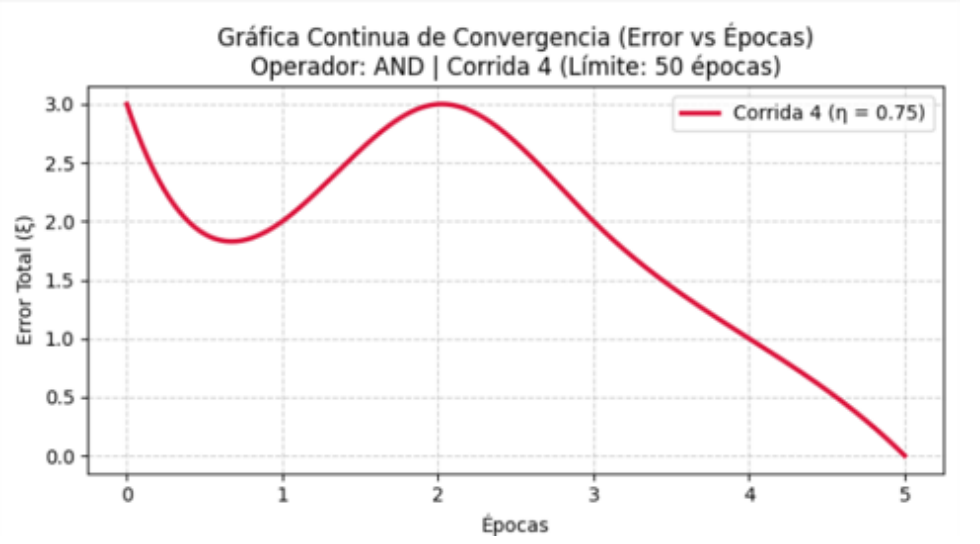
4.2 *Operador OR*

4.2.1 *Corrida 1 (η = 0.01)*

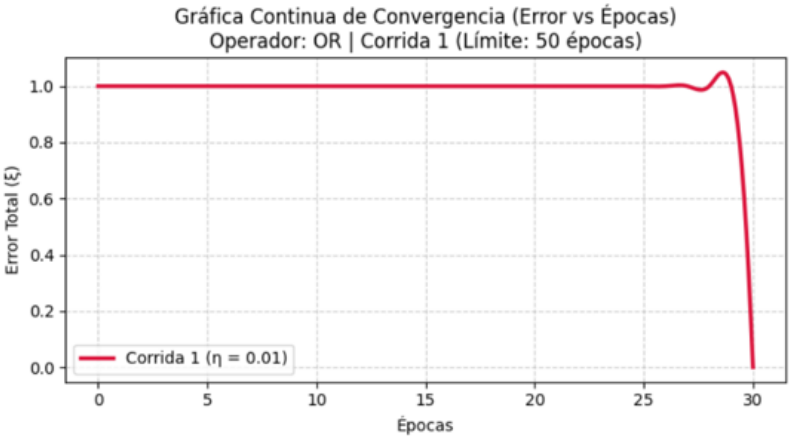
TABLA DE RESULTADOS ACUMULADA - OPERADOR: OR

Corrida	Tasa (η)	Pesos Inic [w0, w1, w2]	Pesos Fin [w0, w1, w2]	Épocas Límite	Épocas Usadas	Estado
1	0.01	[0.2917, 0.5253, 0.1896]	[-0.0083, 0.5253, 0.1896]	50	31	Convergió

- **Pesos Iniciales:** $w_0 = 0.2917, w_1 = 0.5253, w_2 = 0.1896$
- **Pesos Finales:** $w_0 = -0.0083, w_1 = 0.5253, w_2 = 0.1896$



- **Épocas Usadas:** 31 (Límite: 50). Estado: Convergió.



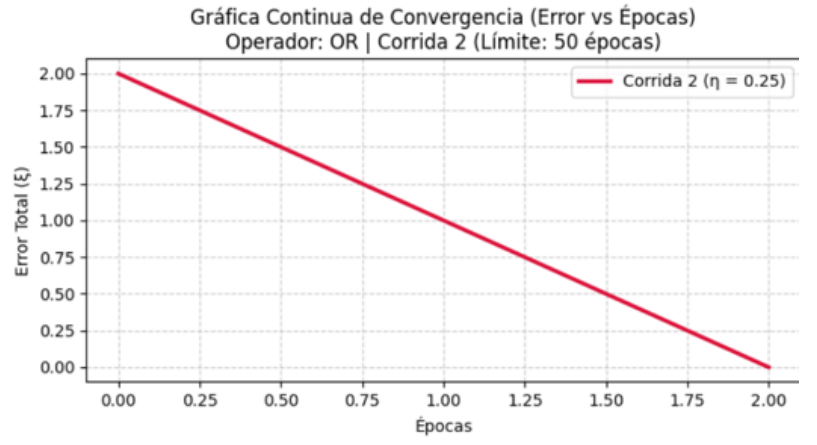
Los incrementos son muy pequeños, manteniendo el error constante durante la mayor parte del proceso hasta lograr la convergencia paulatina en la época 31.

4.2.2 Corrida 2 (η = 0.25)

TABLA DE RESULTADOS ACUMULADA - OPERADOR: OR

Corrida	Tasa (η)	Pesos Inic [w0, w1, w2]	Pesos Fin [w0, w1, w2]	Épocas Límite	Épocas Usadas	Estado
1	0.01	[0.2917, 0.5253, 0.1896]	[-0.0083, 0.5253, 0.1896]	50	31	Convergió
2	0.25	[0.0382, 0.7723, 0.0918]	[-0.2118, 0.7723, 0.3418]	50	3	Convergió

- **Pesos Iniciales:** $w_0 = 0.0382, w_1 = 0.7723, w_2 = 0.0918$
- **Pesos Finales:** $w_0 = -0.2118, w_1 = 0.7723, w_2 = 0.3418$
- **Épocas Usadas:** 3 (Límite: 50). Estado: Convergió.



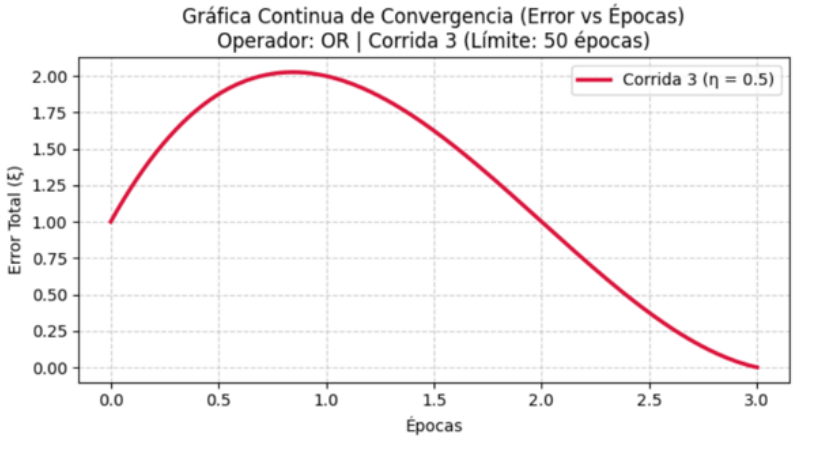
Con una tasa moderada, las correcciones logran una rápida convergencia lineal en tan solo 3 épocas.

4.2.3 *Corrida 3 (η = 0.50)*

TABLA DE RESULTADOS ACUMULADA - OPERADOR: OR

Corrida	Tasa (η)	Pesos Inic [w0, w1, w2]	Pesos Fin [w0, w1, w2]	Épocas Límite	Épocas Usadas	Estado
1	0.01	[0.2917, 0.5253, 0.1896]	[-0.0083, 0.5253, 0.1896]	50	31	Convergió
2	0.25	[0.0382, 0.7723, 0.0918]	[-0.2118, 0.7723, 0.3418]	50	3	Convergió
3	0.50	[0.669, 0.0443, 0.3399]	[-0.331, 0.5443, 0.3399]	50	4	Convergió

- **Pesos Iniciales:** $w_0 = 0.6690, w_1 = 0.0443, w_2 = 0.3399$
- **Pesos Finales:** $w_0 = -0.3310, w_1 = 0.5443, w_2 = 0.3399$
- **Épocas Usadas:** 4 (Límite: 50). Estado: Convergió.



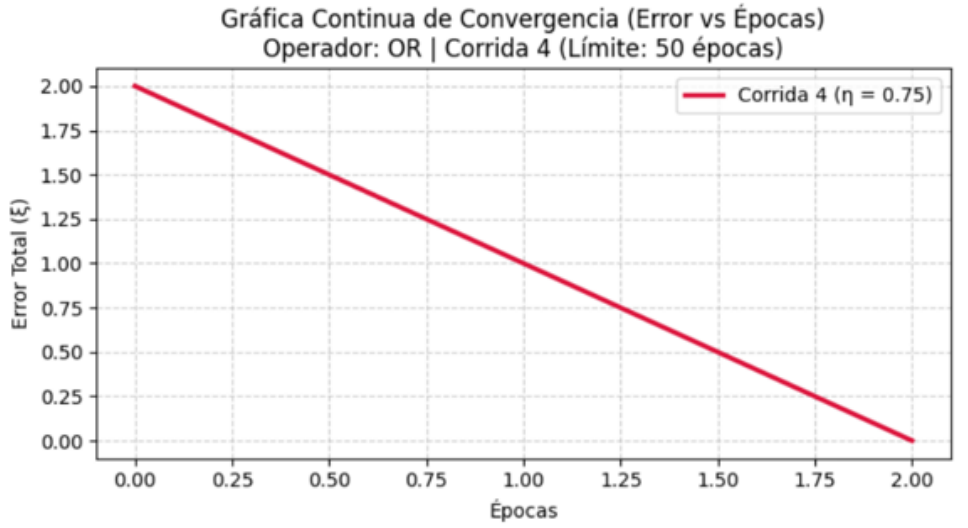
4.2.4 *Corrida 4 (η = 0.75)*

- **Pesos Iniciales:** $w_0 = 0.2568, w_1 = 0.0429, w_2 = 0.5762$
- **Pesos Finales:** $w_0 = -0.4932, w_1 = 0.7929, w_2 = 0.5762$

TABLA DE RESULTADOS ACUMULADA - OPERADOR: OR

Corrida	Tasa (η)	Pesos Inic [w0, w1, w2]	Pesos Fin [w0, w1, w2]	Épocas Límite	Épocas Usadas	Estado
1	0.01	[0.2917, 0.5253, 0.1896]	[-0.0083, 0.5253, 0.1896]	50	31	Convergió
2	0.25	[0.0382, 0.7723, 0.0918]	[-0.2118, 0.7723, 0.3418]	50	3	Convergió
3	0.50	[0.669, 0.0443, 0.3399]	[-0.331, 0.5443, 0.3399]	50	4	Convergió
4	0.75	[0.2568, 0.0429, 0.5762]	[-0.4932, 0.7929, 0.5762]	50	3	Convergió

- **Épocas Usadas:** 3 (Límite: 50). Estado: Convergió.



4.3 Operador NAND

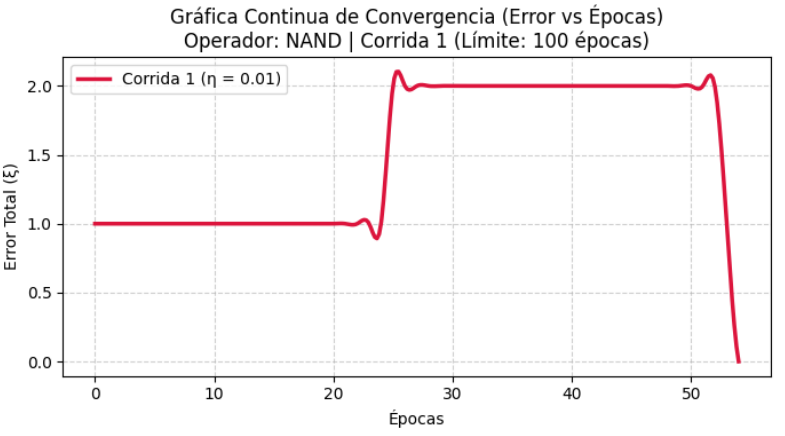
Límite de épocas establecido: 100 épocas por corrida.

4.3.1 Corrida 1 (η = 0.01)

TABLA DE RESULTADOS ACUMULADA - OPERADOR: NAND

Corrida	Tasa (η)	Pesos Inic [w0, w1, w2]	Pesos Fin [w0, w1, w2]	Épocas Límite	Épocas Usadas	Estado
1	0.01	[0.2917, 0.5253, 0.1896]	[0.0517, -0.0047, -0.0504]	100	55	Convergió

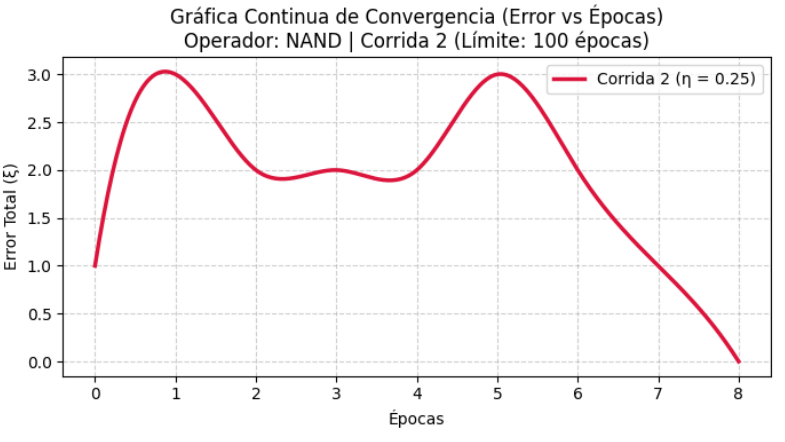
- **Pesos Iniciales:** $w_0 = 0.2917, w_1 = 0.5253, w_2 = 0.1896$
- **Pesos Finales:** $w_0 = 0.0517, w_1 = -0.0047, w_2 = -0.0504$
- **Épocas Usadas:** 55 (Límite: 100). Estado: Convergió.



4.3.2 *Corrida 2* ($\eta = 0.25$)

```
======================================================================================
TABLA DE RESULTADOS ACUMULADA - OPERADOR: NAND
======================================================================================
 Corrida  Tasa (η)  Pesos Inic [w0, w1, w2]     Pesos Fin [w0, w1, w2]  Épocas Límite  Épocas Usadas    Estado
       1      0.01 [0.2917, 0.5253, 0.1896] [0.0517, -0.0047, -0.0504]            100             55 Convergió
       2      0.25 [0.0382, 0.7723, 0.0918] [0.5382, -0.4777, -0.1582]            100              9 Convergió
======================================================================================
```

- **Pesos Iniciales:** $w_0 = 0.0382, w_1 = 0.7723, w_2 = 0.0918$
- **Pesos Finales:** $w_0 = 0.5382, w_1 = -0.4777, w_2 = -0.1582$
- **Épocas Usadas:** 9 (Límite: 100). Estado: Convergió.

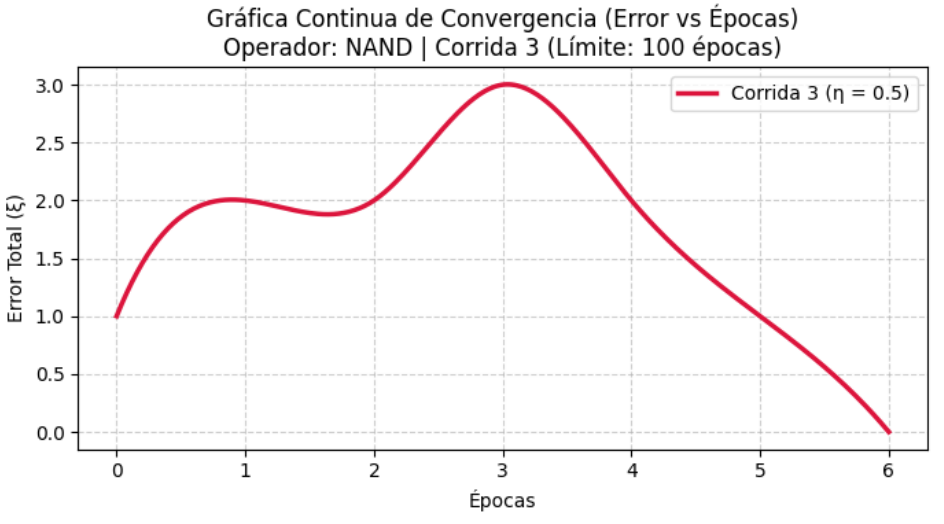


4.3.3 *Corrida 3 ($\eta = 0.50$)*

```
================================================================================
TABLA DE RESULTADOS ACUMULADA - OPERADOR: NAND
================================================================================
 Corrida  Tasa (η)  Pesos Inic [w0, w1, w2]     Pesos Fin [w0, w1, w2]  Épocas Límite  Épocas Usadas    Estado
       1     0.01 [0.2917, 0.5253, 0.1896] [0.0517, -0.0047, -0.0504]            100             55 Convergió
       2     0.25 [0.0382, 0.7723, 0.0918] [0.5382, -0.4777, -0.1582]            100              9 Convergió
       3     0.50  [0.669, 0.0443, 0.3399]  [1.169, -0.9557, -0.6601]            100              7 Convergió
================================================================================
```

- **Pesos Iniciales:** $w_0 = 0.6690, w_1 = 0.0443, w_2 = 0.3399$
- **Pesos Finales:** $w_0 = 1.1690, w_1 = -0.9557, w_2 = -0.6601$
- **Épocas Usadas:** 7 (Límite: 100). Estado: Convergió.

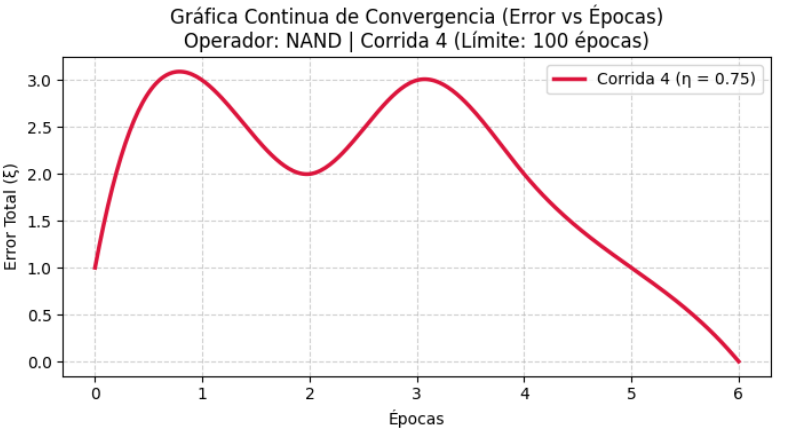
Acelera el cambio de polaridad de los pesos w_1 y w_2, volviéndolos negativos para satisfacer la salida 0 de la entrada $(1, 1, 1)$.



4.3.4 *Corrida 4 ($\eta = 0.75$)*

```
================================================================================
TABLA DE RESULTADOS ACUMULADA - OPERADOR: NAND
================================================================================
 Corrida  Tasa (η)  Pesos Inic [w0, w1, w2]     Pesos Fin [w0, w1, w2]  Épocas Límite  Épocas Usadas    Estado
       1     0.01 [0.2917, 0.5253, 0.1896] [0.0517, -0.0047, -0.0504]            100             55 Convergió
       2     0.25 [0.0382, 0.7723, 0.0918] [0.5382, -0.4777, -0.1582]            100              9 Convergió
       3     0.50  [0.669, 0.0443, 0.3399]  [1.169, -0.9557, -0.6601]            100              7 Convergió
       4     0.75 [0.2568, 0.0429, 0.5762] [1.7568, -1.4571, -0.9238]            100              7 Convergió
================================================================================
```

- **Pesos Iniciales:** $w_0 = 0.2568, w_1 = 0.0429, w_2 = 0.5762$
- **Pesos Finales:** $w_0 = 1.7568, w_1 = -1.4571, w_2 = -0.9238$
- **Épocas Usadas:** 7 (Límite: 100). Estado: Convergió.



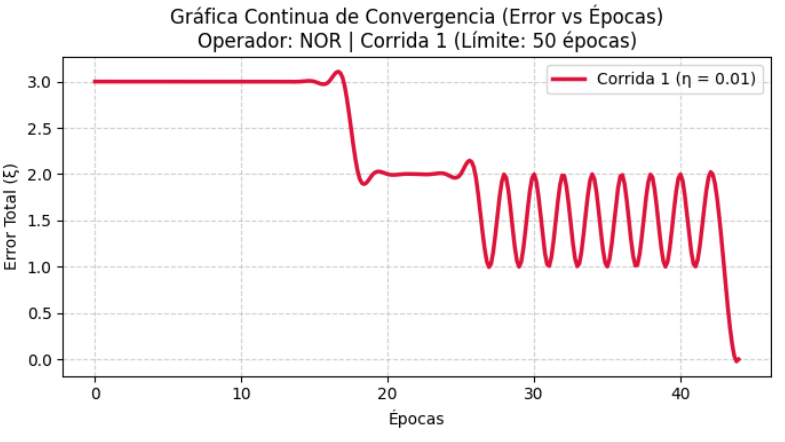
4.4 Operador NOR

Límite de épocas establecido: 50 épocas por corrida.

4.4.1 Corrida 1 (η = 0.01)

```
================================================================================
TABLA DE RESULTADOS ACUMULADA - OPERADOR: NOR
================================================================================
 Corrida  Tasa (η)  Pesos Inic [w0, w1, w2]     Pesos Fin [w0, w1, w2]  Épocas Límite  Épocas Usadas    Estado
       1      0.01 [0.2917, 0.5253, 0.1896] [0.0017, -0.0047, -0.0904]             50             45 Convergió
================================================================================
```

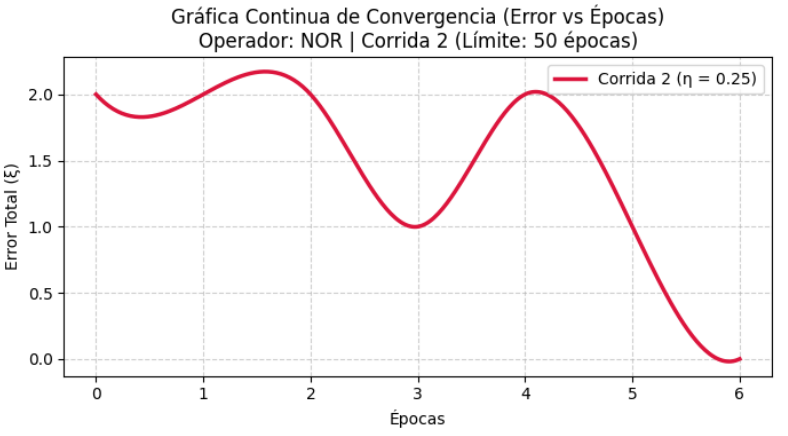
- **Pesos Iniciales:** $w_0 = 0.2917, w_1 = 0.5253, w_2 = 0.1896$
- **Pesos Finales:** $w_0 = 0.0017, w_1 = -0.0047, w_2 = -0.0904$
- **Épocas Usadas:** 45 (Límite: 50). Estado: Convergió.



4.4.2 *Corrida 2 (η = 0.25)*

```
================================================================================
TABLA DE RESULTADOS ACUMULADA - OPERADOR: NOR
================================================================================
 Corrida  Tasa (η)  Pesos Inic [w0, w1, w2]     Pesos Fin [w0, w1, w2]  Épocas Límite  Épocas Usadas    Estado
       1      0.01 [0.2917, 0.5253, 0.1896] [0.0017, -0.0047, -0.0904]             50             45 Convergió
       2      0.25 [0.0382, 0.7723, 0.0918] [0.0382, -0.2277, -0.1582]             50              7 Convergió
================================================================================
```

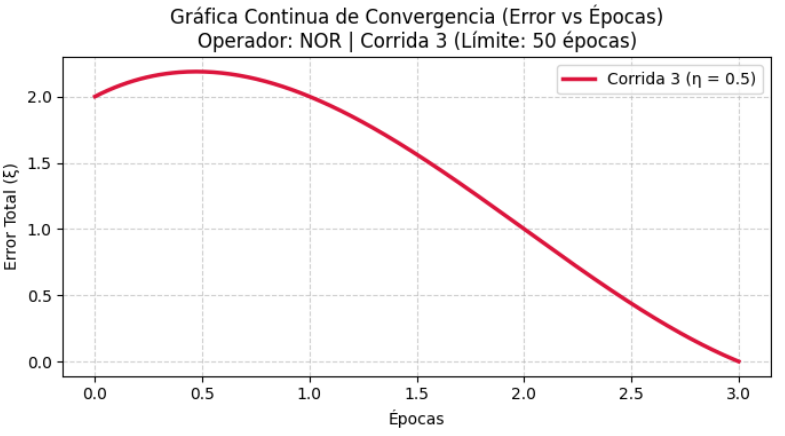
- **Pesos Iniciales:** $w_0 = 0.0382, w_1 = 0.7723, w_2 = 0.0918$
- **Pesos Finales:** $w_0 = 0.0382, w_1 = -0.2277, w_2 = -0.1582$
- **Épocas Usadas:** 7 (Límite: 50). Estado: Convergió.



4.4.3 *Corrida 3 (η = 0.50)*

```
================================================================================
TABLA DE RESULTADOS ACUMULADA - OPERADOR: NOR
================================================================================
 Corrida  Tasa (η)  Pesos Inic [w0, w1, w2]     Pesos Fin [w0, w1, w2]  Épocas Límite  Épocas Usadas    Estado
       1      0.01 [0.2917, 0.5253, 0.1896] [0.0017, -0.0047, -0.0904]             50             45 Convergió
       2      0.25 [0.0382, 0.7723, 0.0918] [0.0382, -0.2277, -0.1582]             50              7 Convergió
       3      0.50  [0.669, 0.0443, 0.3399]  [0.169, -0.4557, -0.6601]             50              4 Convergió
================================================================================
```

- **Pesos Iniciales:** $w_0 = 0.6690, w_1 = 0.0443, w_2 = 0.3399$
- **Pesos Finales:** $w_0 = 0.1690, w_1 = -0.4557, w_2 = -0.6601$
- **Épocas Usadas:** 4 (Límite: 50). Estado: Convergió.

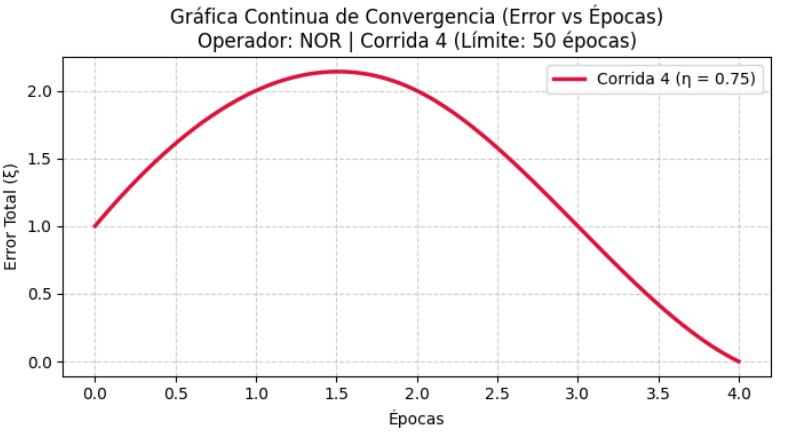


4.4.4 *Corrida 4 (η = 0.75)*

```
================================================================================
TABLA DE RESULTADOS ACUMULADA - OPERADOR: NOR
================================================================================
 Corrida  Tasa (η)  Pesos Inic [w0, w1, w2]     Pesos Fin [w0, w1, w2]  Épocas Límite  Épocas Usadas    Estado
       1     0.01 [0.2917, 0.5253, 0.1896] [0.0017, -0.0047, -0.0904]             50             45 Convergió
       2     0.25 [0.0382, 0.7723, 0.0918] [0.0382, -0.2277, -0.1582]             50              7 Convergió
       3     0.50  [0.669, 0.0443, 0.3399]  [0.169, -0.4557, -0.6601]             50              4 Convergió
       4     0.75 [0.2568, 0.0429, 0.5762] [0.2568, -0.7071, -0.9238]             50              5 Convergió
================================================================================
```

- **Pesos Iniciales:** $w_0 = 0.2568, w_1 = 0.0429, w_2 = 0.5762$
- **Pesos Finales:** $w_0 = 0.2568, w_1 = -0.7071, w_2 = -0.9238$
- **Épocas Usadas:** 5 (Límite: 50). Estado: Convergió.

Ajusta los pesos de entrada fuertemente a valores negativos para garantizar la salida 0 cuando x_1 o x_2 valgan 1.



5. Conclusiones

El desarrollo de este proyecto permitió implementar un modelo de perceptrón capaz de trabajar con diferentes operadores lógicos mediante un proceso de aprendizaje supervisado. A través de las operaciones AND, OR, NAND y NOR fue posible utilizar problemas sencillos para observar de manera práctica el proceso de entrenamiento de una neurona artificial.

La implementación permitió comprobar la importancia de los pesos, la función de activación y la tasa de aprendizaje dentro del proceso de entrenamiento. La incorporación de diferentes tasas permitió establecer un mecanismo para realizar varias corridas y comparar la velocidad de convergencia.

La generación de tablas mediante Pandas y gráficas mediante Matplotlib facilitó la interpretación de los resultados al complementar la información numérica con una representación visual del comportamiento del error.

Finalmente, el uso conjunto de Python, Visual Studio Code, GitHub y LaTeX permitió integrar las diferentes etapas del proyecto, consolidando un flujo de trabajo organizado para la creación y documentación de aplicaciones de inteligencia artificial.

6. Referencias

References

[1] Insente, P. (2020, 10 de marzo). *The ADALINE - Theory and implementation of the first neural network trained with gradient descent*. GitHub. https://pabloinsente.github.io/the-adaline

[2] Universidad del Valle. (s. f.). *[Documento de descarga institucional]*. Programa Editorial, Universidad del Valle. Recuperado el 25 de septiembre de 2026, de https://libros.univalle.edu.co/index.php/programaeditorial/catalog/download/64/28/446?inline=1

[3] El poder del Cómputo. (2022, 8 de agosto). *El modelo de #ia ADALINE paso a paso en #python #inteligenciaartificial #machinelearning* [Vídeo][cite: 23]. YouTube[cite: 23]. https://youtu.be/ZBN8opcCIRM[cite: 23]

[4] Delgado, C. [Carlos Delgado - Computación]. (2018, 13 de septiembre). *Redes Neuronales Clase 4-5 Ejercicio Adaline II* [Vídeo][cite: 24]. YouTube[cite: 24]. https://youtu.be/p0OeKz77hyg[cite: 24]

[5] GeeksforGeeks. (s. f.). *ML - Perceptron implementation in Python*. GeeksforGeeks. Recuperado el 25 de septiembre de 2026, de https://www.geeksforgeeks.org/machine-learning/sklearn-perceptron/

[6] Brownlee, J. (2020, 19 de agosto). *Implement perceptron algorithm from scratch in Python*. Machine Learning Mastery. https://machinelearningmastery.com/implement-perceptron-algorithm-scratch-python/